Optimizavimo metodai

1 Laboratorinis darbas
Vienmatis optimizavimas

Ruošė: Aaron Gandžumian
25-02-2026

- **Tikslas**

  Aprašytą tikslo funkciją

  $f(x) = \frac{(x^2-a)^2}{b} - 1$ (a ir b - studento numerio "2*1**ab" skaitmenys)

  minimizuoti vienmačio optimizavimo algoritmais:
  1. Intervalo dalijimu pusiau
  2. Auksiniu pjūviu
  3. Niutono metodu

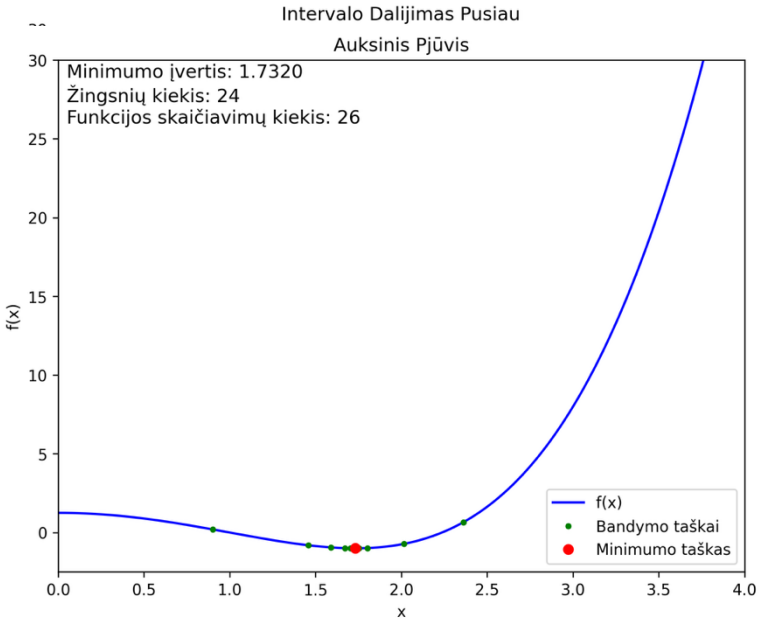
  Palyginti gautus rezultatus: rastas funkcijos minimumo įvertis, atliktų žingsnių ir funkcijų skaičiavimų skaičiai.

- **Skaičiavimai pagal algoritmus**

  Skaičiavimams atlikti tikslo funkcijoje naudojami $a = 3,\ b = 4$.

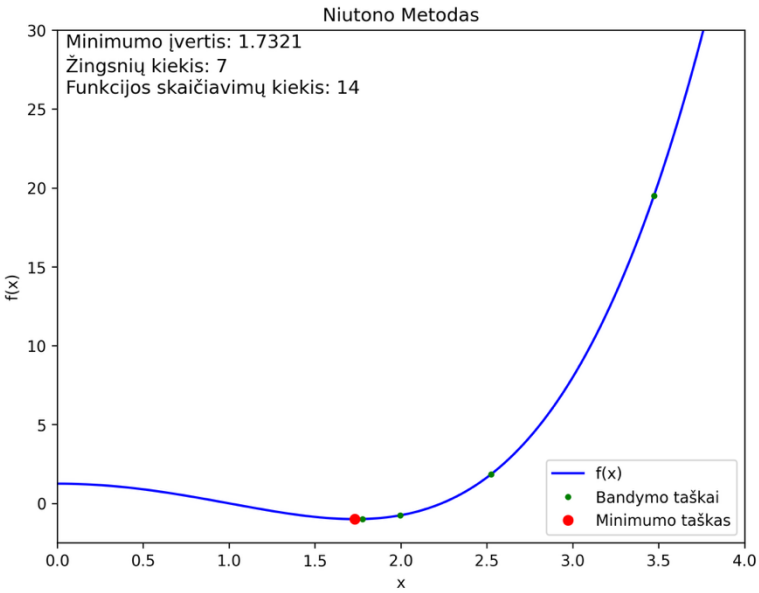
  Tikslo funkcija: $f(x) = \frac{(x^2-3)^2}{4} - 1$

1. Intervalo Dalijimas Pusiau



2. Auksinis Pjūvis

3. Niutono Metodas



- **Išvados ir rezultatai**

Tikslo funkcija yra pakankamai paprasta, kad leistų mums apskaičiuoti tikslųjį minimumo tašką intervale [0, 10] prilyginant jos pirmojo laipsnio išvestinę nuliui.

$$x_{min} = \sqrt{3} \approx 1{,}7321$$

Algoritmas	Minimumo įvertis	Žingsnių kiekis	Funkcijos skaič. kiek.
Intervalo Dalijimas Pusiau	1.7320	17	35
Auksinis Pjūvis	1.7320	24	26
Niutono Metodas	1.7321	7	14

Lygindami algoritmų gautus rezultatus, pastebime, jog **Niutono Metodas** rado tikslo funkcijos minimumo įvertį greičiausiai - per trumpiausią žingsnių bei funkcijos skaičiavimų kiekį.

Kitais atvejais - **Intervalo Dalijimo Pusiau** ir **Auksinio Pjūvio** – algoritmai turi savo privalumų bei trūkumų.

Intervalo Dalijimas Pusiau surado tikslo funkcijos minimumo įvertį atlikdamas mažesnį žingsnių kiekį negu **Auksinis Pjūvis**, tačiau atliko didesnį kiekį funkcijos skaičiavimų.

- **Kodas ir skaičiavimai**

```
from matplotlib import pyplot as plt
import numpy as np

# Parametrai
a = 3
b = 4
N = 10
eps = 0.0001

def func(x):
   return ((x**2 - a)**2 / b) - 1

def deriv_func_1(x):
   return (4 * x * (x**2 - a)) / b

def deriv_func_2(x):
   return (12 * x**2 - 4 * a) / b

# Intervalo dalijimas pusiau
def interval_reduction_method(x_range):
   cycle_count = 0
   func_comp_count = 0
   test_points = []

   l = x_range[0]
   r = x_range[-1]
```

```
    L = r - l
    x_1 = l + L / 4
    x_2 = r - L / 4
    x_m = (l + r) / 2
    f_x_1 = func(x_1)
    f_x_2 = func(x_2)
    f_x_m = func(x_m)
    func_comp_count += 3

    while True:
        test_points.append([x_m, x_1, x_2])

        if f_x_1 < f_x_m:
            r = x_m
            x_m = x_1
            f_x_m = f_x_1
        elif f_x_2 < f_x_m:
            l = x_m
            x_m = x_2
            f_x_m = f_x_2
        else:
            l = x_1
            r = x_2

        cycle_count += 1
        L = r - l
        if(L < eps):
            break

        x_1 = l + L / 4
        x_2 = r - L / 4
        f_x_1 = func(x_1)
        f_x_2 = func(x_2)
        func_comp_count += 2

    x_res = x_m
    y_res = f_x_m
    x_test = np.unique(test_points)
    y_test = func(x_test)

    return x_res, y_res, cycle_count, func_comp_count, x_test, y_test

# Auksinis pjūvis
def golden_section_method(x_range):
    cycle_count = 0
    func_comp_count = 0
    test_points = []
    tau = abs((1 - np.sqrt(5)) / 2)

    l = x_range[0]
    r = x_range[-1]
    L = r - l
```

```
    x_1 = r - tau*L
    x_2 = l + tau*L
    f_x_1 = func(x_1)
    f_x_2 = func(x_2)
    func_comp_count += 2

    while L > eps:
        test_points.append([x_1, x_2])

        if f_x_1 > f_x_2:
            l = x_1
            L = r - l
            x_1 = x_2
            f_x_1 = f_x_2
            x_2 = l + tau*L
            f_x_2 = func(x_2)
        else:
            r = x_2
            L = r - l
            x_2 = x_1
            f_x_2 = f_x_1
            x_1 = r - tau*L
            f_x_1 = func(x_1)

        cycle_count += 1
        func_comp_count += 1

    x_res = (x_1 + x_2) / 2
    y_res = func(x_res)
    x_test = np.unique(test_points)
    y_test = func(x_test)

    return x_res, y_res, cycle_count, func_comp_count, x_test, y_test

# Niutono metodas
def newton_method(x_range):
    x_i = 5
    cycle_count = 0
    func_comp_count = 0
    test_points = []
    step = abs(x_i - x_range[0])

    while step > eps:
        test_points.append(x_i)

        x_new = x_i - deriv_func_1(x_i) / deriv_func_2(x_i)
        step = abs(x_new - x_i)
        x_i = x_new

        cycle_count += 1
        func_comp_count += 2

    x_res = x_i
```

```
    y_res = func(x_res)
    x_test = np.unique(test_points)
    y_test = func(x_test)

    return x_res, y_res, cycle_count, func_comp_count, x_test, y_test

x = np.arange(0, N, step=eps*10)
y = func(x)
func_comp_count = 0
for method_name, method_func in [
    ("Intervalo Dalijimas Pusiau", interval_reduction_method),
    ("Auksinis Pjūvis", golden_section_method),
    ("Niutono Metodas", newton_method)
]:
    plt.figure(figsize=(8, 6))
    plt.ylim(-2.5, 30)
    plt.xlim(0, 4)
    plt.plot(x, y, color='blue', label='f(x)')

    x_res, y_res, cycle_count, func_comp_count, x_test, y_test =
method_func(x)

    plt.plot(x_test, y_test, '.', color='green', label='Bandymo
taškai')
    plt.plot(x_res, y_res, 'o', color='red', label='Minimumo taškas')
    plt.title(method_name)

    plt.xlabel('x')
    plt.ylabel('f(x)')
    plt.legend()
    plt.text(0.05, 26, f"Minimumoa įvertis: {x_res:.4f}\nŽingsnių
kiekis: {cycle_count}\nFunkcijos skaičiavimų kiekis:
{func_comp_count}", fontsize=12)

    filepath = "Project_1/" + method_name.replace(" ", "_") + ".png"
    plt.savefig(filepath, dpi=300, bbox_inches="tight")
    plt.close()
```